

SENTINEL

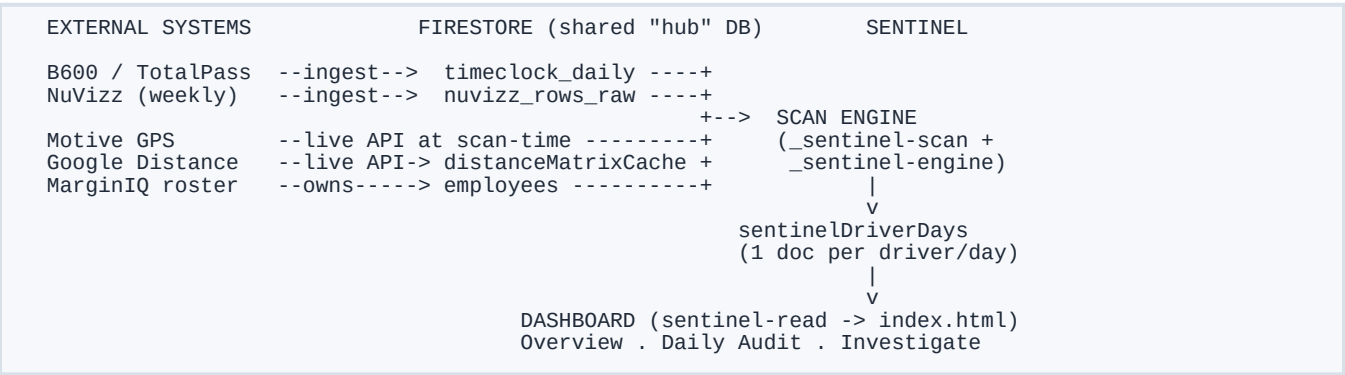
Driver time-theft detection — how it works, how the data is mapped, and what the fallbacks are

Davis Delivery · operator & engineering reference · generated 2026-06-22 · reflects engine v4.5–4.6

One-paragraph summary. SENTINEL takes three independent records of a driver's day — when they clocked in/out (B600 time clock), when they made deliveries (NuVizz), and where the truck physically went (Motive GPS) — and looks for time the driver was on the clock but not accounted for by driving or delivering. It compares the REAL gaps (clock-in -> first delivery, last delivery -> clock-out, and off-route stops mid-day) against an EXPECTED gap (map travel time + a loading/wrap-up allowance). Anything beyond a tolerance is flagged as “unexplained time” and priced in dollars. Everything else on screen is built on that one idea.

1. The big picture (data flow)

SENTINEL does not collect raw data itself. Three upstream systems feed a shared Firestore database; SENTINEL reads from it, scores each driver-day, writes one result document per driver per day, and the dashboard reads those results back.



Critical mental model: the raw collections `timeclock_daily` and `nuvizz_rows_raw` are owned and populated by EXTERNAL ingestion (the B600 scraper / NuVizz weekly import that live in MarginIQ, not in this app). SENTINEL only READS them. If a feed stops landing, SENTINEL has nothing to match — and the day will score “clean / \$0” because there is nothing to compute, not because the driver was clean. This is the single most important thing to understand when a dashboard looks empty.

2. The data sources and who owns what

Source	Firestore collection	Owner	What SENTINEL uses it for
B600 / TotalPass time clock	timeclock_daily	External ingest	Clock-in / clock-out per driver per day. The “on the clock” boundary.
NuVizz delivery system	nuvizz_rows_raw	External ingest	Completed stops with timestamps + addresses. The delivery anchors (first / last).
Motive (GPS / ELD)	(fetched live at scan)	Motive API	Driving periods & destinations to detect off-route / mid-day detours (Class 3).
Google Distance Matrix	distanceMatrixCache	SENTINEL cache	Expected travel time yard <-> first/last stop. Cached per address pair.
Driver roster	employees	MarginIQ	Identity, status, role, truck #, and the cross-system match keys (\$3).

Source	Firestore collection	Owner	What SENTINEL uses it for
Scoring config	sentinelConfig/*	SENTINEL	Defaults (wages, thresholds, load prep), truck-type map, job status.
Results	sentinelDriverDays	SENTINEL	The scored output — one doc per driver per day. Powers the whole UI.
Baselines	sentinelBaselines	SENTINEL	Per-driver “your typical day” context (not used for flag severity).

3. How a driver is mapped across s

This is the heart of “how everything is mapped.” Each system names the same human differently. The `employees` record is the Rosetta Stone: it carries an `externalIds` object plus a truck number, and SENTINEL uses those to join the three feeds.

To match...	Match key on the employee	Matched against	Notes
Time clock punch	<code>externalIds.b600</code>	<code>timeclock</code> row <code>display_id</code> / <code>display_name</code> / <code>payroll_id</code>	Case-insensitive equality; first match wins.
NuVizz deliveries	<code>externalIds.nuvizz</code>	<code>nuvizz</code> row <code>raw["driver name"]</code>	Whitespace-normalized, lower-cased equality.
Motive GPS	<code>externalIds.motive</code>	Motive driver ID (numeric)	Direct ID; no name matching.
Truck type	<code>defaultTruck</code> (truck #)	<code>truckTypeMap.trucks[truck#]</code>	Plus optional per-driver <code>truckType</code> override.

The #1 real-world failure here: the B600 name and the NuVizz name for the same driver are often DIFFERENT (the schema cites “Brent Boyd” vs “Brenton Byrd”). If `externalIds.nuvizz` doesn’t exactly equal the name NuVizz prints, that driver’s deliveries never match — you get clock punches with 0 stops, and every day scores “clean / \$0.” Fixing the driver’s `externalIds.nuvizz` and re-scanning resolves it.

The match-key chain (with fallbacks)

```
B600 punch:  externalIds.b600  -> fullName -> "firstName lastName"
NuVizz stops: externalIds.nuvizz -> fullName
Motive:      externalIds.motive (no fallback; skipped if absent)
Truck type:  employees.truckType -> truckTypeMap[defaultTruck] -> "unknown"
```

4. The scan pipeline (per driver, per day)

For each active driver and a given date, the engine runs these steps (`_sentinel-scan` -> `_sentinel-engine`):

1. Resolve identity & truck type from the `employees` record (§3).
2. Get the clock punch from `timeclock_daily` for that date (multi-punch days use the first IN and last OUT).
3. Get the deliveries from `nuvizz_rows_raw` for that date, keep only Completed stops with a parseable “delivery end” time, sort by time.
4. Window to the shift: drop deliveries before clock-in or after clock-out (recorded as data-health notes, not scored).
5. Pick anchors: first on-shift stop = morning anchor, last = afternoon anchor. A single-stop day still anchors both windows (they don’t overlap, so it isn’t double-counted).
6. Look up travel times `yard` -> first stop and last stop -> `yard` (Distance Matrix, cache first).
7. Score the morning, afternoon, and (if Motive ran) in-route gaps; compute risk score, stolen minutes & dollars (§5).

- Write one document to `sentinelDriverDays/{slug}_{date}`.

5. The scoring model

Each scored “gap” is: actual minutes - expected minutes. Expected = map travel time + a human allowance. Whatever exceeds the “ok” tolerance is unexplained.

Gap (flag class)	Actual	Expected	Unexplained =
Morning (Class 1)	clock-in -> first delivery	travel(yard->first) + load prep	actual - expected, above “ok”
Afternoon (Class 2)	last delivery -> clock-out	travel(last->yard) + wrap up	actual - expected, above “ok”
In-route (Class 3)	off-route minutes between first & last stop (Motive)	0 (any off-route is suspect)	off-route minutes above “ok”
Data integrity (Class 4)	notes only — b600_no_punch, nuvizz_no_stops, no_travel_time_to_first	n/a	not a dollar figure

Thresholds & score weights

Severity	Morning / Afternoon gap	In-route gap	Risk-score points
ok	<= 30 min	<= 15 min	0
warn	<= 60 min	<= 30 min	10
flag	<= 90 min	<= 60 min	25
critical	> 90 min	> 60 min	40

The three class scores are summed and capped at 100, then bucketed:

Risk score	Risk level
0 – 9	clean
10 – 24	low
25 – 44	medium
45 – 69	high
70 – 100	critical

Pricing the time

Stolen minutes (only the portion above the “ok” floor of each gap) convert to dollars at an hourly rate by truck type: $\text{stolenDollars} = (\text{stolenMinutes} \div 60) \times \text{wageRate}[\text{truckType}]$.

Truck type	tractor	straight	unknown
Default wage (\$/hr)	27.50	23.00	25.00

A “self-loader” (load prep >= 60 min) is bucketed separately in the reference metrics, since their expected morning gap is much larger.

6. Fallbacks & defaults — the full list

SENTINEL degrades gracefully: a missing input narrows what can be computed rather than crashing. Every

fallback in the pipeline:

Decision	Preferred	Falls back to	Final fallback
Truck type	per-driver truckType override	truckTypeMap[defaultTruck]	"unknown" (wage 25)
Load prep minutes	employee loadPrepMin	sentinelConfig/defaults	15 min
Wrap-up minutes	employee wrapUpMin	sentinelConfig/defaults	15 min
Wage rate	wageRates[truckType]	wageRates.unknown	25.00 \$/hr
Clock times	punches[] first-IN / last-OUT	top-level clock_in / clock_out	none -> b600_no_punch, no gaps
B600 name match	externalIds.b600	fullName	firstName + lastName
NuVizz name match	externalIds.nuvizz	fullName	none -> no stops matched
Travel time	distanceMatrixCache hit	Google Distance Matrix API	null -> gap not computed
Config doc	stored sentinelConfig/defaults	deep-merged over DEFAULT_DEFAULTS	built-in defaults (no NaN \$)
Yard ZIP set	defaults.yardZips	—	["30518", "30542"]
Motive in-route	Motive driving periods	skipped on backfill / no motive id	inRouteFlag "deferred" (0)
Empty day (nightly)	write only if punch OR stops exist	—	skip write (counted "empty")
Avg clock-in (UI)	circular mean (handles midnight)	—	null if no punches
Unknown risk/ confidence	stored value	coalesced to known bucket	clean / insufficient
Access secret	env SCAN_SECRET	hardcoded literal	should be removed (§11)

Why a day can read "clean / \$0" without anyone being clean. A morning or afternoon gap is computed only when ALL THREE exist: a clock punch, a matched delivery anchor, AND a travel-time estimate. Miss any one and that gap stays null -> contributes 0 -> the day falls through to clean. That is correct (don't accuse on no evidence) but it means \$0 can mean "no evidence," not "no theft." The dashboard now surfaces this difference (§8, §10).

7. What gets stored — sentinelDriverDays

One document per driver per day, ID {driverSlug}_{YYYY-MM-DD}. The fields the dashboard relies on:

Field	Meaning
b600Matched / nuvizzMatched	Did the punch / delivery feed match this driver-day? (The feed-health signal.)
clockIn / clockOut / totalShiftMin	Shift boundaries and length.
firstDeliveryTime / lastDeliveryTime (+addr/ customer)	The morning & afternoon delivery anchors.
completedStops	Count of completed deliveries.
clockInToFirstMin / morningGapMin	Actual morning span / the unexplained portion.
lastToClockOutMin / afternoonGapMin	Actual afternoon span / the unexplained portion.

Field	Meaning
inRouteOffRouteMin / inRouteFlag	Mid-day off-route minutes (Motive) / its severity.
riskScore / riskLevel	Composite 0–100 / its bucket.
stolenMinutes / stolenDollars	The headline unexplained time, priced.
flags[]	Human-readable evidence per flag (the “why this flagged” text).
dataHealth[]	Integrity notes: b600_no_punch, nuvizz_no_stops, no_travel_time_to_first, ...

8. How the KPIs are computed (Overview)

KPI	How it's derived
Unexplained Time (\$ & min)	Sum of stolenDollars / stolenMinutes over every record in the window.
Annualized	Window total × 365 ÷ days in the selected window.
High-risk days / % of fleet	Count of critical+high records ÷ total driver-days.
Risk Distribution	Tally of records by riskLevel.
Top Offenders	Drivers ranked by summed stolenDollars (zero-dollar drivers hidden here).
Reference Metrics	Median morning gap / on-route / etc. per truck-type × pre-load/self-load bucket. Blank medians = no computable gaps.
Baselines	Count of per-driver baseline docs by confidence. Coaching context only.
Feed-health banner (new)	Shows when days were scored but none had a computable gap — names which feed (B600 / NuVizz / travel) is missing. Turns a silent “\$0” into a diagnosis.

Baselines (coaching context, not enforcement): SENTINEL also computes a per-driver “typical day” (median / P75 / P90 of their own history) with a confidence band. Since v4.5 the fraud judgment is a fixed static threshold the same for every driver, so a driver can’t normalize their own theft into an acceptable baseline.

9. Failure modes & how to read them

Because SENTINEL depends on three external feeds, an empty or all-clean dashboard is usually a DATA problem, not a scoring bug. Triage table:

What you see	What it means	What to check
Clock times present, 0 stops, \$0, “no delivery data”	NuVizz feed not matching for that driver/date.	Expand the day -> “Show all deliveries” -> read the diagnostic line.
0 stops AND “no clock punch”	Neither feed landed — total gap for that day.	Coverage probe; nightly scan status.
Whole fleet \$0, every median blank, Baselines still populated	Recent raw feed stale (history exists, recent dates don’t).	Coverage probe — newest NuVizz vs timeclock date.
Punch + stops present but still \$0	Travel-time lookups failing (Distance Matrix / yard).	Confirm SENTINEL_YARD_ADDRESS & GOOGLE_MAPS_API_KEY.

The three diagnostic tools

1. Coverage probe (fleet-wide): `GET /api/sentinel-day-scan?secret=...&coverage=true` returns the newest ingested delivery_date (NuVizz) and date (timeclock). If NuVizz’s newest is older than the window

you're viewing, the weekly import is the problem.

2. Nightly job status: the `sentinelConfig/nightlyScanStatus` doc shows written vs empty vs errors for the last run.
3. Per-day deliveries diagnostic: on any day in Investigate, expand it and click "Show all deliveries for this day - >". The bottom line reports `rowsScannedForDate` and `driverNameMatches`: 0 rows = no NuVizz data ingested for that date (ops/ingestion fix); rows > 0 but 0 matches = the driver's `externalIds.nuvizz` doesn't match the name NuVizz prints (identity fix, then re-scan).

10. Scheduling, endpoints & maintenance

Scheduled jobs

- Nightly scan — daily ~09:00 UTC, targets the date 7 days ago (cushion so the weekly NuVizz email and settled punches have landed). Weekends skipped.
- Weekly baselines — recomputes each driver's "typical day" context.
- Historical backfill — one-time sweep over the full date range (Motive off for speed).
- Re-score all — re-applies current thresholds/config to every stored record without re-fetching raw data (use after changing wages, load-prep, or a driver's truck type).

Key operator endpoints

Endpoint / control	Purpose
<code>/api/sentinel-read?action=dashboard</code>	The whole Overview payload.
<code>/api/sentinel-day-scan?...&coverage=true</code>	Data-freshness probe (newest ingested dates).
<code>/api/sentinel-day-scan?driverSlug=...&date=...</code>	Re-scan a single driver-day (with full debug).
Settings -> Driver Config	Per-driver truck type / load-prep / wrap-up overrides.
Settings -> Re-score All	Apply config changes across history.

11. Security follow-ups

A few items need attention before fully relying on this deployment (full detail in `AUDIT.md`): a Firebase API key is committed in source and must be ROTATED; several endpoints (purge, scan-list delete, introspect) are unauthenticated with open CORS; and the shared access secret has a hardcoded fallback. None of these affect the scoring math, but they should be closed out by an engineer.

This document is a plain-language companion to `DESIGN.md` (design), `SCHEMA.md` (field-by-field contract), and `AUDIT.md` (known issues). Numbers shown are the shipped defaults; per-driver and config overrides take precedence as described in §6.